Ansible et Terraform

CE QUE L'EXAMEN ATTEND

Reconnaître les capacités des outils de gestion de configuration Ansible et Terraform.

01 L'ESSENTIEL

01 Gestion de configuration : l'état voulu est décrit dans des fichiers texte versionnés (Git), puis appliqué de façon reproductible à de nombreux équipements. Idempotence : réappliquer ne change rien si l'état est déjà atteint.

02 Ansible (Red Hat) : sans agent (agentless), depuis un nœud de contrôle, par SSH ou par API ; modèle push. Écrit en Python ; playbooks en YAML.

03 Composants Ansible : inventaire (hôtes et groupes), playbook (suite de plays), play (tâches appliquées à un groupe), tâche (appel d'un module, par exemple `cisco.ios.ios_config`), variables, modèles Jinja2.

04 Terraform (HashiCorp) : infrastructure as code déclarative en HCL ; providers (cloud, Meraki, Catalyst Center…). Cycle : `init`, `plan`, `apply`, `destroy`. Un fichier d'état (state) mémorise ce qui a été créé.

05 Ansible : surtout procédural (ordre des tâches), orienté configuration des équipements. Terraform : déclaratif, orienté provisionnement de ressources.

06 Le blueprint v1.1 cite Ansible et Terraform ; Puppet et Chef (agents, modèle pull) n'y figurent plus.

ANSIBLE · ntp.yml (YAML)

```
- name: Configurer NTP ---------- play
  hosts: routeurs ---------- groupe de l'inventaire
  gather_facts: no
  tasks:
    - name: Serveur NTP ---------- tâche
      cisco.ios.ios_config: ---------- module
        lines:
          - ntp server 192.0.2.123
```

nœud de contrôle → SSH → équipements, sans agent

TERRAFORM (HCL)



FIG. 40 À gauche, un play qui pousse un serveur NTP au groupe routeurs. À droite, le cycle Terraform : décrire, prévisualiser, appliquer.

02 COMMANDES CLÉS

COMMANDE	RÔLE
`$ ansible-inventory -i hosts.yml --list`	VÉRIF. Affiche l'inventaire
`$ ansible-playbook -i hosts.yml ntp.yml --check`	VÉRIF. Simulation, sans changement
`$ ansible-playbook -i hosts.yml ntp.yml`	Exécute le playbook
`$ terraform init`	Télécharge les providers
`$ terraform plan`	VÉRIF. Aperçu des changements
`$ terraform apply`	Applique l'état voulu
`$ terraform destroy`	Supprime les ressources gérées

03 PIÈGES D'EXAMEN

P1 Ansible n'installe aucun agent sur les équipements ; Puppet et Chef en exigent un.

P2 Playbook Ansible en YAML ; configuration Terraform en HCL (fichiers .tf).

P3 `terraform plan` ne modifie rien : seul `apply` applique.

04 AUTO-TEST

1 Quelle caractéristique décrit Ansible ?

A. Agent sur chaque équipement **B.** Sans agent, via SSH
C. Langage HCL **D.** Modèle pull

2 Quelle commande Terraform affiche les changements sans les appliquer ?

A. terraform init **B.** terraform apply
C. terraform plan **D.** terraform destroy

3 Dans quel format s'écrit un playbook Ansible ?

A. JSON **B.** HCL **C.** XML **D.** YAML

REPÈRE Ansible : YAML et SSH, sans agent. Terraform : HCL, plan puis apply.

RÉPONSES **1 B** Ansible pousse ses tâches par SSH ou API depuis un nœud de contrôle.
2 C plan compare l'état voulu à l'état réel et liste les écarts.
3 D Playbooks et inventaires sont en YAML (ou INI pour l'inventaire).